

✓ AI Assignment 13: Introduction to Generative AI and GPT Text Generation

generation using a pre-trained GPT model.

GitHub Repository:

<https://github.com/Mohammedharkan/ai-assignment13-generative-ai-gpt>

1. Introduction

Generative Artificial Intelligence is a type of AI that can create new content such as text, images, code, audio, and videos. In this assignment, I will explore the basics of Generative AI with a focus on GPT models. I will use a pre-trained GPT model to generate text and test different generation settings such as output length, temperature, top-k, top-p, and prompt engineering.

✓ 2. GPT Architecture

Generative Pre-trained Transformers (GPTs) are based on the Transformer architecture, which uses a self-attention mechanism to understand relationships between words in a sentence. Unlike older sequential models, transformers process all input tokens simultaneously, making training faster and more efficient.

The text generation process starts with tokenization, where the input text is divided into tokens and converted into numerical IDs. These tokens are passed through the GPT model, which predicts the probability of the next token based on the previous context. This process is repeated one token at a time until the desired output length is reached.

GPT models are pre-trained on very large text datasets, allowing them to learn grammar, language patterns, and contextual relationships. During text generation, parameters such as temperature, Top-k, and Top-p influence how the next token is selected, producing outputs that range from predictable to creative.

```
# Install required library
!pip install transformers torch -q
```

```
# Import required libraries

import torch
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import pandas as pd
import matplotlib.pyplot as plt

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

✓ 2. Model and Tokenizer Loading

In this section, I load a pre-trained GPT-2 model and its tokenizer from the Hugging Face Transformers library. The tokenizer converts text into numerical tokens, while the model uses these tokens to generate meaningful text.

```
# Load GPT-2 tokenizer and model

model_name = "distilgpt2"

tokenizer = GPT2Tokenizer.from_pretrained(model_name)

model = GPT2LMHeadModel.from_pretrained(model_name)

print("Model loaded successfully!")
print("Tokenizer loaded successfully!")
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
 WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en

tokenizer_config.json: 100%	26.0/26.0 [00:00<00:00, 543B/s]
vocab.json: 100%	1.04M/1.04M [00:00<00:00, 13.7MB/s]
merges.txt: 100%	456k/456k [00:00<00:00, 12.9MB/s]
tokenizer.json: 100%	1.36M/1.36M [00:00<00:00, 16.6MB/s]
config.json: 100%	762/762 [00:00<00:00, 31.6kB/s]
model.safetensors: 100%	353M/353M [00:03<00:00, 224MB/s]
Loading weights: 100%	76/76 [00:00<00:00, 2221.30it/s]
generation_config.json: 100%	124/124 [00:00<00:00, 5.54kB/s]

Model loaded successfully!
 Tokenizer loaded successfully!

3. Text Generation Function

In this section, I create a reusable function that generates text using the pre-trained GPT model. The function takes a text prompt as input and returns generated text based on different generation parameters.

```
# Text Generation Function

def generate_text(prompt,
                  max_length=50,
                  temperature=1.0,
                  top_k=50,
                  top_p=0.95):

    # Convert text into tokens
    inputs = tokenizer(prompt, return_tensors="pt")

    # Generate text
    outputs = model.generate(
        **inputs,
        max_length=max_length,
        temperature=temperature,
        top_k=top_k,
        top_p=top_p,
        do_sample=True,
        pad_token_id=tokenizer.eos_token_id
    )

    # Convert tokens back to text
    generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return generated_text
```

4. Exercise 1: Basic Text Generation

In this exercise, I use the pre-trained GPT model to generate text from a simple prompt. This demonstrates the model's ability to continue a sentence based on the context provided.

```
# Exercise 1: Basic Text Generation

prompt = "Artificial Intelligence is"

generated_text = generate_text(prompt)

print("Prompt:")
print(prompt)

print("\nGenerated Text:")
print(generated_text)

Prompt:
Artificial Intelligence is
```

Generated Text:

Artificial Intelligence is a powerful and promising new kind of intelligence that can be used by a wide variety of organizations

Observation

The model successfully generated text based on the prompt. The output is related to Artificial Intelligence, but it stopped before completing the final sentence because the default maximum length was limited. This shows how output length affects text generation.

5. Exercise 2: Controlling Output Length

In this exercise, I compare text generation using two different output lengths. This demonstrates how the maximum length parameter affects the amount of generated text.

```
# Exercise 2: Controlling Output Length
```

```
prompt = "Artificial Intelligence is"
```

```
print("==== Short Output =====\n")
```

```
print(generate_text(prompt, max_length=30))
```

```
print("\n\n==== Long Output =====\n")
```

```
print(generate_text(prompt, max_length=100))
```

```
==== Short Output =====
```

```
Artificial Intelligence is still in the works, but it.s far, far away. Its not like its really
```

```
==== Long Output =====
```

```
Artificial Intelligence is, to be clear, only a limited number of companies and services use artificial intelligence to make the
```

```
"You must provide support and resources to companies that require AI to conduct an AI research program that has the potential to
```

Observation

Increasing the value of `max_length` allows the model to generate longer and more detailed text. A smaller value produces shorter responses, while a larger value gives the model more space to continue the generated content.

6. Exercise 3: Adjusting Temperature

In this exercise, I compare text generation using different temperature values. Lower temperatures produce more predictable text, while higher temperatures create more diverse and creative outputs.

```
# Exercise 3: Adjusting Temperature
```

```
prompt = "Artificial Intelligence is"
```

```
print("==== Temperature = 0.3 =====\n")
```

```
print(generate_text(prompt, temperature=0.3))
```

```
print("\n\n==== Temperature = 1.0 =====\n")
```

```
print(generate_text(prompt, temperature=1.0))
```

```
print("\n\n==== Temperature = 1.5 =====\n")
```

```
print(generate_text(prompt, temperature=1.5))
```

```
==== Temperature = 0.3 =====
```

```
Artificial Intelligence is a new technology that has been used to create a new kind of artificial intelligence.
```

```
===== Temperature = 1.0 =====
```

Artificial Intelligence is a system designed to detect the presence of a human brain during long-term emotional processing and r

```
===== Temperature = 1.5 =====
```

Artificial Intelligence is using Artificial Intelligence to learn from everyday life around us using real-time cognitive skills

Observation

Lower temperature values produce more predictable and focused text. Higher temperature values generate more creative and diverse responses, but the text may become less consistent.

7. Exercise 4: Using Top-k and Top-p

In this exercise, I compare text generation using different values of Top-k and Top-p. These parameters control how the model selects the next word by limiting the candidate words based on either the number of choices or the cumulative probability.

```
# Exercise 4: Using Top-k and Top-p
```

```
prompt = "Artificial Intelligence is"
```

```
print("===== Top-k = 20 =====\n")
print(generate_text(prompt, top_k=20))
```

```
print("\n\n===== Top-k = 80 =====\n")
print(generate_text(prompt, top_k=80))
```

```
print("\n\n===== Top-p = 0.70 =====\n")
print(generate_text(prompt, top_p=0.70))
```

```
print("\n\n===== Top-p = 0.95 =====\n")
print(generate_text(prompt, top_p=0.95))
```

```
===== Top-k = 20 =====
```

Artificial Intelligence is a computer-generated image (also called an image of an object) that is physically present in the imag

```
===== Top-k = 80 =====
```

Artificial Intelligence is one of the reasons, by far, I prefer to talk about the Internet as a kind of research tool for intell

What makes this possible is that this technology may make

```
===== Top-p = 0.70 =====
```

Artificial Intelligence is also a key component of intelligence, and we know it has a wide range of applications. But what makes

===== Top-p = 0.95 =====

Artificial Intelligence is at a very low level at the moment, but it appears that artificial intelligence is doing some things a

So why do we have so many people?
In one way,

Observation

Lower Top-k and Top-p values produce more focused and predictable text because the model selects from fewer candidate words. Higher values increase diversity by allowing the model to consider a wider range of possible words during text generation.

8. Exercise 5: Prompt Engineering

In this exercise, I use different prompts to demonstrate how the quality and style of the generated text depend on the prompt provided to the model.

```
# Exercise 5: Prompt Engineering

prompts = [
    "Write a short story about a robot.",
    "Explain machine learning in simple words.",
    "Write a professional email to a customer about a delayed order."
]

for i, prompt in enumerate(prompts, 1):
    print(f"\n===== Prompt {i} =====")
    print("Prompt:")
    print(prompt)

    print("\nGenerated Text:")
    print(generate_text(prompt))

    print("\n" + "="*60)
```

```
===== Prompt 1 =====
Prompt:
Write a short story about a robot.

Generated Text:
Write a short story about a robot.
```

```
The last year I'd seen a story about a robot named "arobot."
This was just another story about a robot called the "robot." I started thinking about all robots
```

```
=====
```

```
===== Prompt 2 =====
Prompt:
Explain machine learning in simple words.
```

```
Generated Text:
Explain machine learning in simple words. Using data from the first two paragraphs, researchers at the University of Waterloo an
```

```
=====
```

```
===== Prompt 3 =====
Prompt:
Write a professional email to a customer about a delayed order.
```

```
Generated Text:
Write a professional email to a customer about a delayed order. We are not able to determine what is the best use of the email a
```

=====

Observation

Different prompts produce different types of responses. Clear and specific prompts help the model generate more relevant and useful text, demonstrating the importance of prompt engineering in Generative AI applications.

9. Methodology and Findings

In this assignment, I used a pre-trained DistilGPT2 model from the Hugging Face Transformers library. The tokenizer converted the input prompts into numerical tokens, and the GPT model generated new text based on these tokens.

The experiments showed that different generation parameters affect the output. Increasing `max_length` produced longer responses. Lower `temperature` created more predictable text, while higher `temperature` produced more creative but sometimes less consistent text. Top-k and Top-p helped control the range of candidate words used during generation. Prompt engineering also showed that clear and specific prompts can improve the relevance of generated text.

11. Ethical Considerations

Generative AI has many benefits, but it also raises ethical concerns. These models may generate incorrect information, biased content, or text that sounds realistic but is not true. They may also be misused to create spam, fake news, or misleading content.

Possible solutions include reviewing AI-generated content before using it, clearly labeling AI-generated work, using reliable datasets, reducing bias during model development, and following privacy and copyright rules.

12. Conclusion

This assignment helped me understand the basics of Generative AI and GPT-based text generation. I learned how GPT models use tokenization, probabilities, and sequence generation to create text. I also explored how parameters such as output length, temperature, Top-k, Top-p, and prompt engineering affect the generated output. In the future, Generative AI can support many areas such as education, business, customer service, and content creation, but it must be used responsibly.